

ERflow

Technical System Documentation & Architecture Guide

Project: ERflow — Emergency Department Decision Support Prototype

Team: Argo (Karan Aditya, Jai A Mishra)

Institution: Indian Institute of Technology (IIT) Guwahati

Deployment: Local-first — all inference runs on-device; no patient data leaves the machine

Executive Summary: ERflow is a two-stage decision support system for Emergency Department (ED) patient prioritization. Stage 1 assesses each incoming patient with one of three age-specific XGBoost risk models trained with an asymmetric (false-negative-averse) loss, bounds the result with a deterministic ESI Version 5 safety floor, explains it with SHAP attributions and a tree-level confidence score, and writes a grounded clinical narrative using a local Gemma 4 model served by Ollama. Stage 2 places the patient in a continuously re-scored priority queue with full clinician override and audit logging.

1. Core Principles

- **Asymmetric Risk Optimization:** missing a critical patient (false negative) is far costlier than over-prioritizing a stable one, so training penalizes false negatives 18–28× more than false positives.
- **Deterministic Safety Floors:** $\text{Final_ESI} = \min(\text{ML_ESI_Recommendation}, \text{ABCDE_ESI_Floor})$. Rules can only escalate a patient; the model can never downgrade one below the floor.
- **Zero-History Resilience:** about half of ED arrivals have no verified records, so the dataset and models are built with a 50% zero-history share and rely on presenting vitals and frailty.
- **Human-in-the-Loop:** low-confidence predictions trigger a mandatory review gate, and nurses can override any acuity level with a recorded rationale.
- **Local-First Privacy:** models, SHAP, rules and the language model all run on the local machine.

2. Synthetic Training Dataset (scripts/generate_data.py)

Models are trained on a reproducible synthetic ED cohort produced by `generate_synthetic_ed_data()` (the same generator is prototyped in `ERflow_Modelling.ipynb`). Default size is **6,230 patients** with random seed **42**, so every run yields identical data.

Component	Generation Logic
Age cohorts	Pediatric 15% (ages 1–17), Adult 55% (18–64), Geriatric 30% (65–94). Realised: 970 / 3,431 / 1,829.
Gender	Female 47%, Male 53%.
Clinical Frailty Scale	Pediatric fixed at 1. Adult CFS 1–4 ($p = 0.50/0.30/0.15/0.05$). Geriatric CFS 1–9 ($p = 0.05/0.15/0.25/0.20/0.15/0.10/0.05/0.03/0.02$).
Prior history	50% verified records, 50% zero-history. Comorbidity count $\sim \text{Poisson}(\lambda = 2.5 \text{ geriatric}, 0.8 \text{ otherwise})$ when history exists, else 0.
Baseline acuity (ESI v4)	ESI 1–5 drawn with $p = 0.023 / 0.271 / 0.417 / 0.270 / 0.019$.
Vital signs	Generated from the baseline acuity around cohort baselines (adult HR 75, RR 16, SBP 120; geriatric HR 72, RR 18, SBP 135; pediatric HR 110/90, RR 28/20 below/above age 5, SBP $90 + 2 \times \text{age}$). ESI 1 draws unstable extremes; ESI 3 and 4 carry an 18% / 10% chance of one abnormal vital. Values are clipped to HR 30–220, RR 4–60, SpO2 60–100, SBP 50–240, Temp 34–41.5 °C.
ESI v5 level	ESI 3–5 patients with any age-specific danger-zone vital (HR, RR or SpO2 < 92%) are escalated to ESI 2.

Component	Generation Logic
Outcome label	critical_outcome = ICU admission OR 30-day mortality. ICU probability 0.65 / 0.12 / 0.03 (+0.02 if CFS ≥ 5) / 0.005 for ESI 1 / 2 / 3 / 4–5; mortality 0.225 / 0.031 (+0.03 if CFS ≥ 5) / 0.016 (+0.02 if CFS ≥ 5) / 0.001. Result: 536 positives (8.6%).
Queue features	Wait time ~ Exponential(scale 10 min for ESI 1–2, 45 min otherwise), tripled for the 25% of patients on peak-hour shifts. Also records resources used and override occurrence.

3. Demographic XGBoost Model Training (scripts/train_models.py)

A single adult-calibrated model is unsafe across age groups because physiological baselines differ, so ERflow trains three independent gradient-boosted models, one per cohort. Each cohort is split 80/20 into train and test sets, stratified on the outcome label (random_state = 42).

3.1 Training Configuration

Setting	Value
Library / API	XGBoost native API: xgb.train() on xgb.DMatrix inputs
Boosting rounds	150
Tree parameters	max_depth = 4, learning_rate = 0.01, tree_method = 'hist', min_child_weight = 1
Objective	Custom asymmetric logistic loss (below); built-in eval metric disabled (disable_default_eval_metric = 1)
Decision threshold	t = 0.504 on P_risk = sigmoid(raw margin), shared by all three models
Export	Booster saved as JSON: models/geriatric_xgb.json, adult_xgb.json, pediatric_xgb.json; evaluation summary written to models/models_metadata.json

3.2 Asymmetric Logistic Objective

For label $y \in \{0, 1\}$ and $p = 1 / (1 + e^{-\text{margin}})$, the loss is $L = -[\alpha \cdot y \cdot \log(p) + \beta \cdot (1 - y) \cdot \log(1 - p)]$ with $\beta = 1.0$. XGBoost receives its first and second derivatives:

```
grad = p * (alpha * y + beta * (1 - y)) - alpha * y
hess = p * (1 - p) * (alpha * y + beta * (1 - y))
```

3.3 Per-Cohort Models and Held-Out Test Performance

Model	α	Rows (train / test)	Critical	Features	AUC	Sens.	Spec.	TP / FN / FP / TN
Geriatric (65+)	23.0	1,829 (1,463 / 366)	158	HR, RR, SpO2, SBP, Temp, CFS, prior history, comorbidity count	0.728	87.5%	49.4%	28 / 4 / 169 / 165
Adult (18–64)	18.0	3,431 (2,744 / 687)	298	HR, RR, SpO2, SBP, Temp, prior history, comorbidity count	0.805	88.3%	53.4%	53 / 7 / 292 / 335
Pediatric (<18)	28.0	970 (776 / 194)	80	Age, HR, RR, SpO2, SBP, Temp, prior history, comorbidity count	0.695	75.0%	52.2%	12 / 4 / 85 / 93

α = false-negative weight in the loss; Critical = patients with the critical outcome; AUC = ROC-AUC; Sens. / Spec. = sensitivity / specificity at $t = 0.504$ on the held-out test set.

Test-set figures are recomputed from the shipped model files and match models_metadata.json. The operating point deliberately trades specificity for sensitivity: across the three test sets 93 of 108 critical patients are flagged. Retrain with `python -m erflow.scripts.train_models`; because the dataset is seeded, retraining reproduces these models.

4. Inference, Uncertainty & Explainability

- **Cohort routing:** the patient's age cohort field selects the model; if absent, age < 18 → pediatric, age ≥ 65 → geriatric, otherwise adult. Missing inputs default to CFS 2, prior history 0, comorbidities 0.
- **ML recommendation (agent/tools.py):** $P_risk \geq 0.504 \rightarrow$ ESI 2 (escalate); otherwise ESI 3 (standard).
- **Tree-level confidence (explain/uncertainty.py):** P_risk is re-predicted using only the first 15, 30, ... 150 trees. The standard deviation of these 10 stage predictions is normalised by 0.25 and mapped to a 0–100% confidence score. Below 20% the patient is flagged for mandatory human review.
- **SHAP attribution (explain/shap_explainer.py):** shap.TreeExplainer computes exact Shapley values on the cohort's model; the top 5 features by |SHAP| are returned with direction and age-banded normal ranges from data/esi_reference_tables.json (infant, toddler/young child, school age/adolescent, adult, geriatric).
- **Grounding validator (explain/grounding.py):** every narrative is checked before display — claimed ESI must not be less urgent than the safety floor, each cited risk factor must match its SHAP sign, cited values must match intake data within ± 0.5 , cited features must exist in the model's feature space, and cited rules must have actually fired. Any violation marks the narrative ungrounded and forces human review.

5. Deterministic ESI v5 Safety Floor Engine (rule_engine/)

Decision Point	Clinical Logic	Acuity Floor
A (decision_a.py)	Immediate life-saving intervention, cardiac arrest / apneic / pulseless, intubation or agonal respiration / airway compromise, AVPU = U or GCS ≤ 8, HR < 30 or > 220, SBP < 50.	ESI 1 (hard locked)
B (decision_b.py)	High-risk situation, altered mental status (AVPU V/P or GCS 9–13), severe pain/distress (or pain ≥ 8/10 with systemic distress), ischemic chest pain, stroke symptoms (FAST+), evolving anaphylaxis, acute suicidal ideation.	ESI 2
D (decision_d.py)	Danger-zone vitals — infant <1 y: HR > 160, RR > 45; age 1–4: HR > 140, RR > 35; age 5–17: HR > 110, RR > 24; all pediatric: SpO2 < 92%, fever ≥ 38 °C under 3 months. Adult/geriatric: HR > 100, RR > 20, SpO2 < 92%, SBP < 90. Geriatric frailty guard: age ≥ 65, CFS ≥ 5 and HR > 90.	ESI 2
C (decision_c.py)	Expected resources: 2+ → ESI 3, 1 → ESI 4, 0 → ESI 5. If resources were not collected, returns insufficient_data and defaults to ESI 3.	ESI 3 / 4 / 5

6. Local Language Model: Gemma 4 via Ollama (agent/llm_client.py)

The clinical narrative shown in Stage 1 is written by a local Gemma 4 model served by Ollama. The model never decides acuity — it only explains results already computed by the models and rules, and its output is validated by the grounding validator before display.

Setting	Value
Default model	gemma4:12b (override with the OLLAMA_MODEL environment variable)
Server	http://localhost:11434 (override with OLLAMA_HOST); endpoint POST /api/chat
Availability check	One TCP connect to 127.0.0.1:11434 with a 50 ms timeout when the client starts
Request options	stream = false, format = "json", temperature = 0.1, think = false (passed inside options)
Timeout	15 seconds per request
Input	User message = JSON of the patient record, ML result, rule result, SHAP result and final ESI
Output handling	message.content is parsed as JSON and returned as the structured narrative (provider = ollama)

System prompt sent with every request:

You are ERflow, a clinical decision support assistant for emergency department patient prioritization. Synthesize a factual, grounded clinical acuity assessment based strictly on the provided ML predictions, SHAP feature rankings, and ESI v5 deterministic rules. Do not hallucinate. Return a structured JSON summary.

6.1 Deterministic Fallback

If Ollama is not running, the request fails, times out or returns invalid JSON, ERflow switches automatically to a built-in deterministic narrative engine (provider = deterministic_clinical_engine, model = esi_v5_grounded_synthesizer), so the system works fully offline with no setup. The fallback emits the fields the dashboard and validator consume: clinical_rationale, alignment_status, primary_risk_factors (feature, value, direction, SHAP impact), triggered_safety_rules, risk and confidence percentages, and the review flag.

6.2 Tool Schemas

agent/tools.py also exposes Ollama-compatible function schemas for four tools — predict_esl_xgboost, get_shap_explanation, run_esl_rule_engine and lookup_reference_range. In the current pipeline the orchestrator calls these tools directly in Python and passes their results to the model in a single request.

6.3 Setup

```
ollama pull gemma4:12b      # download the model once
ollama serve                # start the local server on port 11434
python main.py --ui         # ERflow detects the server at startup
```

7. Assessment Pipeline (agent/orchestrator.py)

AssessmentOrchestrator.analyze_patient() runs these steps for every patient:

- Run the cohort's XGBoost model → P_risk, ML ESI, confidence score, review flag.
- Run the ESI v5 rule engine → acuity floor and triggered rules.
- Apply the safety floor: Final_ESI = min(ML ESI, floor); record the agreement state (AGREEMENT, RULE_SAFETY_ESCALATION or ML_PREDICTIVE_ESCALATION).
- Compute SHAP attributions (top 5) and the Stage 2 priority score.
- Generate the narrative (Gemma 4 via Ollama, or the deterministic fallback) and validate it; an ungrounded narrative forces human review.

8. Stage 2: Queue Scheduler (scheduler/)

```
Priority Score(t) = W_floor x (6 - ESI_final) + W_risk x P_risk + W_time x ln(1 + t_wait / tau)
W_floor = 1000   W_risk = 100   W_time = 15   tau = 30 min
```

- **Tier preservation:** each ESI level is worth 1,000 points, while risk adds at most 100 and waiting adds about 36 points after 5 hours, so a lower-acuity patient can never overtake a higher-acuity one.
- **Ordering:** scores are recalculated on every read and the queue is ranked with a heap (heapq on negated scores); the top entry is the next patient called to a bed.
- **Time advance:** staff enter any number of minutes to advance all waiting times and re-score the queue.
- **Overrides and audit:** a nurse override sets a new ESI, re-scores and re-sorts immediately, and appends an audit entry (timestamp, patient, clinician ID, original and new ESI, reason, resulting score).

9. Application Interfaces

9.1 Streamlit Dashboard (ui/app.py)

Three tabs: **Stage 1: Patient Intake & Assessment** (case presets, intake form, results with SHAP chart, rules and grounded narrative), **Stage 2: Queue Scheduler** (live ranked queue, demo population, custom time advance, bed assignment, nurse override) and **Nurse Override Audit Log**. Theme settings live in .streamlit/config.toml.

9.2 FastAPI Service (api/main.py)

Endpoint	Purpose
GET /health	Service status, loaded models and queue size
POST /analyze-patient	Full Stage 1 assessment without changing the queue
GET /queue	Ranked queue and override count
POST /queue/add	Assess a patient and add them to the queue
POST /queue/advance-time	Advance all wait times by 0.1–120 minutes
POST /queue/pop-next	Call the highest-priority patient to a bed
POST /override	Record a nurse override and re-sort
POST /queue/populate-demo	Fill the queue with 1–50 synthetic patients

10. Verification Suite (19 tests)

Test Module	Scope	Tests
test_rule_engine.py	Decision Points A–D, frailty guard, resource fallback, min(ML, floor) bounding	7
test_scheduler.py	Priority formula, ESI tier preservation, intra-tier risk ordering, time advancement, queue ranking, deterioration re-sort, override and pop	5
test_grounding.py	Grounded narratives, inverted-SHAP rejection, floor-breach rejection, numeric fidelity	4
test_api.py	Health and analysis endpoints; queue add, advance-time, override and pop-next lifecycle	3

11. Environment & Commands

Python 3.13 with xgboost 3.4, shap 0.52, scikit-learn 1.9, numpy 2.5, pandas 3.0, streamlit 1.62, altair 6.2, fastapi 0.141, uvicorn 0.52, pydantic 2.13, requests 2.34 and pytest 9.1.

Command	Action
python main.py --ui	Launch the dashboard at http://localhost:8501
python main.py --api	Launch the REST API at http://localhost:8000 (docs at /docs)
python main.py --simulate	Assess 10 synthetic patients in the terminal
python main.py --test	Run the 19-test verification suite
python -m erflow.scripts.train_models	Regenerate the dataset and retrain all three models
python -m erflow.scripts.generate_documentation_pdf	Rebuild this document

Decision Support Notice: ERflow provides risk estimates, safety floors and grounded explanations to support clinicians. Final acuity decisions and clinical disposition always remain with the attending healthcare professional, who can override any recommendation.